Loading stimuli labels and MUA data from MATLAB files

Al Xin

May 21, 2026

1 Stimulus labels

The table of stimuli information contains the family ID, coherence, sample ID, and block placement. Each unique combination of family ID, coherence, and sample ID corresponds to a specific image, and each image is repeated four times in the session. Blank images are arbitrarily labeled with family ID `0` and sample ID `id_blank` from 1 to 100.

Family ID, coherence, and sample ID columns are identical across all files and only need to be saved once. However, there is a column marking the position of the stimulus within the block, which is unique for each session file. I cannot find the order of the blocks in the published data of the blocks. This setup does not generate unique rows because the repetition number of each unique image (defined by the combination of coherence, family, and sample ID) is not recorded in any column and block placement is arbitrary and may be repeated. I assume the repetition number was implicitly captured by the row number, but I am not confident this is the scheme Lee et al. intended.

1.1 Column details

- `coherence`: `0` when blank, `1` when noisy, `2` when naturalistic.
- `family`: `0` when blank, two or three digit integer code otherwise.
- `id`: `0` when blank, integer from `1` to `15` otherwise.
- `id_blank`: integer from `1` to `100` if blank, `0` otherwise.
- `repetition`: column constructed from counting the repetition of `coherence`, `family`, and `id` or `id_blank` combo.

```
# get arbitrary identical stimulus labels
stims_ref <- list.files(t2_dir) %>%
  .[1] %>% # arbitrary index
```

```
  paste0(t2_dir, .) %>%
  R.matlab::readMat() %>%
  .$save.label %>%
  data.frame()
colnames(stims_ref) <- c(
  "coherence", "family", "id", "coherence_t3", "x5", "id_blank")
stimuli <- select(stims_ref, -coherence_t3, -x5) %>%
  group_by(coherence, family, id, id_blank) %>%
  mutate(repetition = row_number()) %>%
  ungroup()
```

2 Binned multiunit activity

Reading from MATLAB tends to be time-consuming, so it is easier to work with the binned multiunit activity (MUA) pre-extracted and saved in R data files with the quick serialization package `qs2`. The locations of `NA` values were removed. I manually recoded the subject IDs to be consistent with the monkey IDs in the table and arbitrarily labeled the arrays `a` or `b` with the special cases of `a` being shared for the V1/V2 border array on M1 and `c` used for the second implantation of M6 (Table 1). This separated the 17 original session files into 27 subject files.

Furthermore, though the published MATLAB files contain the subject IDs, there does not appear to be a mapping of the binned data to the subject. Again, I assume that the mapping is implicitly captured by the ordering and split the binned data into ordered chunks of 96 sites.

```
data.frame(ada_id = id_recoding) %>%
  tibble::rownames_to_column("lee_id") %>%
  relocate(lee_id, .before = ada_id) %>%
  gt() %>%
  tab_options(table.font.size = px(12)) %>%
  cols_label(
    lee_id = "Lee et al. IDs",
    ada_id = "IDs in ADA")
```

```
n <- 96 # number of channels
files <- list.files(t2_dir)

for (f in files) {
  f_details <- file_info(f)
  dat <- paste0(t2_dir, "/", f) %>% R.matlab::readMat()
```

Table 1: Original MATLAB subject IDs from Lee et al. (left) and new IDs for this project (right).

Lee et al. IDs	IDs in ADA
KY_n1	M1a
KY_v1	M1a
KY_n2	M1b
LI_n1	M2a
LI_n2	M2b
BB_n1	M3a
BB_n2	M3b
HT_V4	M6a
HT_IT	M6b
HT_n2	M6c
HU_n2	M7a
XT_n2	M8a

```
  subj <- as.character(dat$incl.subj)
  bins <- dat$bins

  # start messaging block
  cli_text(paste0(
    "Session {f_details$region} at {f_details$age} weeks", " ",
    "({f_details$n_subjects} subjects)"))
  ulid <- cli_ul()

  # walk through subjects in file
  for (i in 1:length(subj)) {
    # get the more informative subject code
    id_code <- id_recoding[subj[i]]
    # incorporate into file name (qs2 extension)
    f_name <- paste0(
      f_details$region, "_",
      f_details$age, "_",
      id_code, ".qs2")
    cli_li("{i}/{length(subj)}: saving {subj[i]} to {f_name}...")
    # message(i, "/", length(subj),  ": ", subj[i], " to ", f_name, "...")

    # manually subset the MUA data by number of channels
    idx_last <- i * n
    idx_first <- idx_last - n + 1
    bins_ <- bins[, , idx_first:idx_last]

    # detect missing channels in 3rd dimension
```

```
    missing_sites <- is.na(bins_[1, 1, ])
    bins_ <- bins_[, , !missing_sites]
    if (sum(missing_sites) > 0)
      # message("NOTE: ", sum(missing_sites), " missing channels.")
      cli_alert_info("Found {sum(missing_sites)} missing sites.")

    qs_save(bins_, paste0(mua_dir, f_name))
  }
  # close messaging block
  cli_end(ulid)
  cli_rule()
}
```